XPtoIRL: Installation and User Manual

Release 1.0.0 | Project: Software Engineering (DLMCSPSE01) | Leela Sai Abhinav Gamidi (UPS10617430)

1 Overview

XPtoIRL is a web application that recommends video games for the real-life skills a user wants to develop. The application can be used in three ways: online in the cloud (no installation), locally with Docker (recommended), or locally with Python only.

2 Online use

Open the link given in the submission file `XPtoIRL_links.txt` in a current browser (Chrome, Firefox or Edge). The file also contains the login data of a test administrator. On the free cloud plan the first request after a longer pause can take up to one minute, because the service is started again.

3 Installation with Docker (recommended)

Requirements: Docker Desktop with Docker Compose, Git.

```
git clone <repository URL>
cd xptoirl
docker compose up --build
```

Open `http://localhost:8000`. On the first start the container creates the PostgreSQL tables and imports the data set (17 skills, 66 games, 175 mappings, 26 sources). If port 8000 is already used, choose another host port:

```
# macOS / Linux
WEB_PORT=8010 docker compose up --build
# Windows PowerShell
$env:WEB_PORT=8010; docker compose up --build
```

To create an administrator, set `ADMIN_EMAIL` and `ADMIN_PASSWORD` in a `.env` file (copy `.env.example`) before the start, or run:

```
docker compose exec web flask create-admin admin@example.org
```

Stop the application with `docker compose down`; add `-v` to delete the database as well.

4 Installation without Docker

Requirements: Python 3.11 or newer. Without `DATABASE_URL` the application uses a local SQLite file.

```
python -m venv .venv
.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements-dev.txt
set FLASK_APP=wsgi.py           # macOS/Linux: export FLASK_APP=wsgi.py
flask db upgrade
flask seed
flask create-admin admin@example.org
flask run
```

Open `http://127.0.0.1:5000`.

5 Cloud deployment (Render)

The repository contains the Render Blueprint `render.yaml`. In the Render dashboard choose **New** > **Blueprint**, select the GitHub repository, enter `ADMIN_EMAIL` and `ADMIN_PASSWORD` and confirm. Render creates the PostgreSQL database and the web service, generates `SECRET_KEY`, sets `DATABASE_URL` and checks the service with `/api/health`. Each push to the main branch deploys the new version.

6 Configuration

Variable	Meaning	Default
`APP_ENV`	`development`, `production` or `testing`	`development`
`SECRET_KEY`	Key for sessions and CSRF tokens, required in production	dev value
`DATABASE_URL`	SQLAlchemy URL of the database	SQLite file
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Administrator created on container start	empty
`WEB_PORT`	Host port used by `docker compose`	8000
`PORT`	Port of Gunicorn inside the container (set by the cloud)	8000

7 Using the application

1. **Find games.** On the start page choose one to three skills (grouped by cognitive, social, motor and emotional) and press *Find games*.
2. **Filter.** Narrow the results by platform, genre, PEGI age, price model, play mode and session length. Each result shows the score, the reason, the evidence level (A strong, B medium, C weak) and the sources.

3. **Game page.** Open a game to see all trained skills with reasons and links to the studies. The search field finds games by title.
4. **Learning plan.** After registering, save games to the learning plan, rate how well a game trained a skill (1 to 5) and print the plan. Under *Account* the account can be deleted together with all data.
5. **Administration.** Administrators open *Admin* to create, edit and delete skills, games, sources and mappings, or to import the four CSV files. Every mapping needs at least one source.

8 Tests and quality checks

```
pytest --cov=app                                    # 86 tests
ruff check .                                        # linter
python scripts/load_test.py --base-url http://localhost:8000 --users 20
```

GitHub Actions runs the linter, the tests with a coverage gate (service layer $\geq$ 70 %) and a Docker build for every push.

9 Troubleshooting

Problem	Solution
port is already allocated	Start with another `WEB_PORT` (Section 3).
Docker cannot connect to the daemon	Start Docker Desktop and wait until it shows “running”.
SECRET_KEY must be set in production	Set `SECRET_KEY` or use `APP_ENV=development` locally.
Empty start page	Run `flask seed` (or restart the container) to import the data set.